

BancOS - Kernel

Ivan Calvo

2026-01-09

Contents

1. Introducción	2
1.1. BancOS.	2
1.2. Estructura del proyecto (Archivos).	3
2. Arquitectura del sistema.	4
2.1. Diseño.	4
2.2. Principales estructuras de datos.	5
2.3. Scheduler/Dispatcher.	8
2.4. Concurrencia y sincronización.	9
3. Manual de uso.	10
3.1. Archivos de configuración.	10
3.2. Parámetros de lanzamiento.	10
3.3. Consola de Comandos.	11
4. Conclusiones.	12



1. Introducción

Puedes encontrar el proyecto en <https://github.com/pixelpercebe/BancOS>

Este informe detalla el diseño e implementación de **BancOS**, un simulador de kernel de sistema operativo desarrollado en lenguaje C estándar (ANSI C). Este proyecto aborda las Fases 1 (Arquitectura del Sistema) y 2 (Planificador) de la práctica de la asignatura, estableciendo un entorno de ejecución concurrente multihilo y un sistema de planificación de procesos avanzado.

1.1. BancOS.

El kernel BancOS simula los componentes hardware esenciales (CPU, Cores, Reloj, Timer) mediante el uso de hilos (**pthreads**) y mecanismos de sincronización (**Mutex** y **variables de condición**). También dispone de una consola interactiva con la que poder comunicarte directamente para obtener la información del kernel en cualquier momento de la ejecución.

1.1.1. La Filosofía BancOS.

BancOS a diferencia de los sistemas operativos traducionales tipo UNIX, que buscan equidad entre procesos, se establece como una metáfora económica basada en el capitalismo.

En BancOS el tiempo de CPU no es un derecho del proceso si no un recurso del mercado. Los procesos no son simples tareas del sistema operativo, son agentes económicos clasificados en diferentes estratos sociales (Vagabundo, pobre, Clase media, Élite).

En este entorno, como es la realidad capitalista, la función del kernel es maximizar el “retorno de inversión” del sistema, priorizando aquellos procesos que “aportan” más al sistema (mayor presupuesto o estatus), bajo la política denominada Capital Gains Scheduling (CGS).

1.1.2. Propósito de BancOS.

El propósito de BancOS es tanto crear un motor de simulación estable capaz de gestionar la creación aleatoria de carga y la ejecución concurrente sin condiciones de carrera ni Deadlocks. Se busca conseguir una Eficiencia en la Planificación, implementando un Scheduler capaz de tomar decisiones en tiempo constante ($O(1)$) gracias al uso de estructuras de datos optimizadas (listas doblemente enlazadas intrusivas, bitmaps y hasmaps). Además el diseño está enfocado en la simplificación de la configuración de lanzamiento del sistema mediante el uso de archivos de configuración .ini y comandos que dan al usuario libertad para guardar y cargar diferentes configuraciones en cualquier momento.

1.2. Estructura del proyecto (Archivos).

Cuando observamos la estructura de archivos del desarrollo de BancOS podemos ver los siguientes ficheros ordenados de manera modular para facilitar la mantenibilidad, compilación y división de responsabilidades.

- **bin/**: Contiene el binario final ejecutable del simulador una vez que ha sido compilado exitosamente.
- **build/**: Aquí se guarda el **Makefile** principal y es la ubicación donde se generan y almacenan los archivos de registro (*logs*) resultantes de cada compilación.
- **config/**: Almacena los archivos de persistencia de configuración en formato **.ini**. El sistema requiere obligatoriamente la existencia del archivo **config.ini**, que actúa como la configuración por defecto, si el usuario no especifica un archivo personalizado por línea de comandos.
- **docs/**: Documentación integral del proyecto. Contiene este informe técnico de diseño y la guía de referencia de comandos para el usuario final.
- **include/**: Contiene todos los archivos de cabecera (**.h**), los cuales se han separado para diferenciar los enfocados a el hardware y configuración de la máquina de la lógica del sistema operativo:
 - *Subdirectorio machine/*: Cabeceras orientadas a la configuración y definición del hardware emulado.
 - *Subdirectorio OS_task/*: Cabeceras orientadas a las tareas fundamentales del kernel, como el Generador de Procesos o el Scheduler (Llamados TAKS en BancOS).
 - *Raíz*: Definiciones globales críticas como **pcb.h** o **os.h**.
- **src/**: Contiene el código fuente de la implementación (archivos **.c**), donde está la lógica correspondiente a las definiciones de las cabeceras.
- **tools/**: Scripts de utilidad y automatización. Incluye el script encargado de la compilación y gestionar el guardado organizado de los *logs* en el directorio **build**.

La gestión de carga de trabajo sigue el patrón clásico de productor-consumidor: **Productor (Process Generator)** Crea nuevos PCBs con atributos aleatorios (tiempo de vida, clase social) e intenta insertarlos en la cola global del sistema **Consumidor (Scheduler)**: Extrae procesos de la cola para asignarlos a la CPU. **Recurso Crítico**: Dado que ambos hilos acceden a la misma estructura de datos (la lista de procesos), el diseño implementa también mutex.

2.2. Principales estructuras de datos.

Los datos se agrupan en diferentes estructuras de datos.

Hardware

```
typedef struct {
    u_int thread_id;
    PCB *current_process;
    u_int is_idle;
    // Otros campos específicos de core.h
} HardwareThread;

typedef struct {
    int core_id;

    pthread_t thread_id;        // El hilo padre (scheduler) del sistema operativo
    pthread_mutex_t lock;       // Mutex para sincronización del core
    pthread_cond_t wake_cond;   // Señal de "Despertador" (Tick)
    int should_work;            // Bandera: 1 = Trabaja, 0 = Duerme

    // Configuración económica local
    int current_rent_price;     // Precio de alquiler actual del core

    // Recursos hardware (Hilos/Pisos)
    // Puntero para soportar número variable de hilos por core
    HardwareThread *threads;
    int num_configured_threads; // Número de hilos configurados en este core

    // Cola de espera local (RunQueue)
    // lista de los procesos esperando piso en ESTE core
    // (estruct RunQueue en scheduler.h)
    struct RunQueue *run_queue;
    int force_eviction;         // Bandera para indicar si hay una interrupción pendiente
} Core;
```

Estos structs guardan la información de los cores y los hilos. El proceso current process del hilo, es el proceso actual en ejecución.

```
typedef struct timer
{
    int timer_id;
    int duration;
    int is_active;
    void (*callback)(void);
} Timer;
```

Estructura utilizada para la gestión de interrupciones de software. Vincula un identificador y una duración con una función de retorno (callback), permitiendo que el hilo *Clock* ejecute rutinas específicas (como despertar al Scheduler) cuando expira un contador, sin bloquear el flujo principal.

```
typedef struct{
    u_int num_cpu;
    u_int cpu_num_cores;
    float cpu_clock_speed_Ghz;
    u_int cpu_hardware_threads;
    u_int cpu_cache_L1;
    u_int cpu_cache_L2;
    u_int cpu_cache_L3;
    char * replacement_policy;
    u_int scheduler_tick_freq;
    u_int total_cores; //no se puede calcular en tiempo de compilacion
    u_int total_hardware_threads; //no se puede calcular en tiempo de compilacion
} Machine;
```

Actúa como el descriptor de configuración del hardware. Almacena parámetros estáticos definidos al inicio (frecuencia de reloj, frecuencias...) y parámetros calculados dinámicamente como el número total de hilos de hardware (*total_hardware_threads*) y núcleos (*total_cores*). Esta estructura es consultada por el hilo *Clock* para gestionar los ciclos de ejecución.

Gestión global del sistema

```
typedef struct system_t{

    Machine machine_data;
    PCB *list_pcb_head; // Cabeza de la lista global
    PCB *list_pcb_tail; // Cola de la lista global
    int process_count; // Contador actual
    pthread_mutex_t list_pcb_lock; // Mutex para proteger ESTA lista

    // -----
    u_int number_of_tasks; // número de tareas cargadas
    Core *cores; // Array dinámico de cores
    u_int bucket_cgs_granularity; // Granularidad de los buckets CGS
    u_int max_budget; // Presupuesto máximo para un proceso
    Timer task_map[TASK_LIST_LENGTH]; // Mapa de tareas del sistema
    int verbose_mode; // Modo verbose activado/desactivado
} system_t;
```

La estructura *system_t* funciona como el objeto contenedor principal (*Singleton*) del kernel, centralizando el estado de la simulación.

- **Gestión de Memoria:** Contiene los punteros `list_pcb_head` y `list_pcb_tail` que definen la *lista global de todos los procesos vivos en el sistema*, protegida por un cerrojo dedicado (`list_pcb_lock`) para garantizar la seguridad entre hilos frente a accesos concurrentes del Generador y el Scheduler.
- **Recursos:** Mantiene el array dinámico de `Cores` y los parámetros de configuración de la política económica (CGS), como el presupuesto máximo (`max_budget`).

PCB de procesos

```
typedef struct PCB{
    int pid;                // ID del proceso
    int lifetime;
    int final_tick;
    SocialClass class;
    int last_core;          // id Último core donde se ejecutó
    int last_thread;        // id Último hilo donde se ejecutó
    ProcessState state;     // Estado del proceso (READY, RUNNING, BLOCKED, etc.)
    int budget;
    struct PCB *next_br;    // Puntero al siguiente PCB en la lista de todos los procesos
    struct PCB *prev_br;    // Puntero al PCB anterior en la lista de todos los procesos
    struct PCB *next_pcb_runqueue; //proximo puntero en la runqueu del bucket
    struct PCB *prev_pcb_runqueue; //proximo puntero en la runqueu del bucket
} PCB;
```

La estructura PCB en BancOS esta diseñada para soportar una doble vinculación, permitiendo que un proceso pertenezca a múltiples estructuras lógicas simultáneamente sin duplicar los datos:

1. **Punteros Globales (`next_br`, `prev_br`):** “Brother pointers”. Mantienen al proceso en la lista global del sistema (en `system_t`). Permiten la eliminación segura en $O(1)$ cuando el proceso finaliza.
2. **Punteros de Planificación (`next_pcb_runqueue`, `prev_pcb_runqueue`):** Permiten que el Scheduler mueva el proceso entre las distintas colas de prioridad (Buckets) sin afectar a su posición en la lista global.
3. **Datos Económicos:** Almacena la `SocialClass` y el `budget`, que son los criterios para la política de planificación.

```
typedef struct Bucket {
    PCB *head;
    PCB *tail;
} Bucket;
```

// Estructuras del Scheduler

```
typedef struct RunQueue {
    Bucket *buckets; //array dinámico de punteros a PCB
    u_int * active_bitmap; // bitmap de buckets activos
    u_long bitmap_size; // Tamaño del array de ints del bitmap
    u_int count;       // número de procesos en la cola
    u_int num_buckets; // número de buckets
    u_int max_active_bucket; // bucket más alto que tiene procesos
} RunQueue;
```

El Scheduler utiliza una arquitectura de colas multinivel optimizada, *inspirada en el planificador $O(1)$ de Linux*:

- **Bucket:** Representa un nivel de prioridad o “estrato económico”. Es una lista simple que agrupa procesos con un rango de presupuesto similar. Se puede regular mediante la granularidad.
- **RunQueue:** Es la estructura maestra de planificación. En lugar de una lista única, mantiene un array dinámico de `buckets`.
 - **Optimización de Bitmap:** Incluye un campo `active_bitmap` que marca qué buckets tienen procesos listos. Esto permite al Scheduler encontrar el proceso con mayor presupuesto (prioridad) mediante operaciones a nivel de bit, evitando recorrer colas vacías y garantizando una complejidad algorítmica constante independientemente del número de procesos vivos en el sistema.

2.3. Scheduler/Dispatcher.

2.3.1. Capital Gains Scheduling.

En el sistema CGS el tiempo de CPU se gestiona como un recurso valioso de mercado.

Los procesos compiten por este recurso utilizando un “presupuesto” virtual, donde la prioridad es directamente proporcional a su riqueza acumulada.

El sistema se fundamenta en tres características: clases sociales, Planificación por tramos fiscales $O(1)$ (buckets) y Dinámica de Gasto.

1. clases sociales y Presupuestos Cada proceso nace con una clase definida en su PCB: su `SocialClass`. Esta clase determina las condiciones iniciales y los límites económicos del proceso:

- **Elite y Alta:** La elite de la sociedad, procesos críticos con alto presupuesto inicial y acceso a quantums de ejecución extendidos aunque gastan mucho más presupuesto en ejecución.
- **Media y Pobre:** Procesos estándar con presupuestos moderados.
- **Vagabundo:** Procesos de fondo (*background*) que solo se ejecutan cuando el sistema tiene ciclos ociosos y casi no gastan presupuesto.

El campo `budget` del PCB también actúa como una **prioridad dinámica** entre procesos de la misma clase social. A mayor presupuesto, mayor urgencia de ejecución.

2. Arquitectura de Colas $O(1)$ (Buckets & Bitmap)

Para garantizar que el Scheduler mantenga un rendimiento constante independientemente del número de procesos activos, se ha evitado ordenar la cola de procesos (operación $O(N \log N)$). En su lugar, se ha implementado una estructura de Array de Buckets indexada por presupuesto.

- **Tramos fiscales (Buckets):** El rango de presupuesto posible se divide en segmentos (definidos por `bucket_cgs_granularity`). con una granularidad de 100, un proceso con presupuesto 85 irá al Bucket 0, uno con presupuesto 1000 al 1...
- **Bitmap de Actividad (`active_bitmap`):** Para evitar recorrer el array de buckets buscando uno que no esté vacío, se mantiene un mapa de bits. Cada bit representa un bucket. Si el bit es 1, el bucket tiene procesos; si es 0, está vacío, además el algoritmo de selección de CGS utiliza instrucciones a nivel de bit (bitwise operations):
 - Encontrar el primer bit activo en `active_bitmap` (usando la instrucción `__builtin_clz` para contar ceros).
 - Acceder directamente al bucket correspondiente en el array.
 - Extraer el primer proceso (`head`) de ese bucket.

Esto garantiza una complejidad $O(1)$ en la selección del proceso que tiene que entrar a ejecutarse.

3. El Algoritmo del Ciclo Económico El flujo de vida de un proceso bajo CGS sigue los siguientes pasos:

- **Selección:** Se elige el proceso del bucket más alto (más rico).
- **Ejecución (Gasto):** El proceso utiliza la CPU. Por cada *tick* de reloj consumido, su **budget** disminuye. Es el “precio” de usar el procesador.

La tasa a pagar total sigue la formula de

$$TasaPorTick_{ClaseSocial} + (QuantumBase_{ClaseSocial} \cdot Demanda)$$

Esta formula responde a la demanda, que depende del numero de procesos esperando a entrar en la cpu. Cuantos mas procesos haya mas rapido cambiara de ciontexto el hilo.

- **Desahucio (Re-scheduling):** Al finalizar su turno (o ser interrumpido), su presupuesto ha bajado. El Scheduler calcula su nuevo bucket (ahora inferior) y mueve el proceso a esa nueva cola.
- **Bancarrota y Rescate:** Si un proceso agota su presupuesto, cae al bucket más bajo (prioridad mínima). Para evitar la inanición (*starvation*) definitiva, el sistema implementa un mecanismo de salarios por clase social periódico que inyecta presupuesto base a los procesos empobrecidos, permitiéndoles volver a competir.

En resumen **CGS** esta diseñado como un *hashmap de colas enlazadas* con interrupciones. Cada key del hashmap es un tramo fiscal, y el valor es una lista enlazada de procesos, que se eligen mediante round robin. La accion de crear un nuevo proceso genera una interrupcion.

Como se observa, el sistema CGS aunque eficiente en la mayoría de casos, castiga a la clase social mas baja o los procesos mas empobrecidos siendo un reflejo directo del mismo capitalismo.

2.4. Concurrencia y sincronización.

Para cumplir con el requisito de simular un entorno multiprocesador real, BancOS utiliza la biblioteca **POSIX Threads (pthread)**. El sistema no es una ejecución secuencial, sino hilos independientes (Clock, Timer, Generator, Scheduler) trabajando en paralelo.

Esta arquitectura obliga a implementar mecanismos estrictos de sincronización para proteger la integridad de los datos compartidos:

Exclusión Mutua (Mutex): Se han definido cerrojos (`pthread_mutex_t`) para proteger las “secciones críticas”. entre los más relevante esta el mutex del reloj, para sincronizar los timers, los mutes de los cores para acceder a las Runqueue, y el `list_pcb_lock`, que protege la lista global de procesos. Sin estos procesos de cerrojos y condiciones el funcionamiento del sistema seria completamente impredecible

Variables de Condición: Para evitar la espera activa que consumiría CPU real innecesariamente, se utilizan variables de condición (`pthread_cond_t`). Esto permite que hilos como el *Scheduler* permanezcan dormidos (bloqueados) hasta que reciben una señal explícita (signal) del *Timer* indicando que ha ocurrido un “tick” del sistema.

3. Manual de uso.

3.1. Archivos de configuración.

BancOS da la opción de cargar archivos de configuración en formato .ini para poder hacer test o ejecutar el kernel de diferente manera.

Estos archivos de configuración deben mantener una estructura con dos partes bien diferenciadas y deben especificarse todos los campos aunque después sean sobrescritos mediante comandos. El orden de los campos de configuración es variable.

```
[machine]
num_cpu = 1
cpu_num_cores = 2
cpu_clock_speed_Ghz = 1
cpu_hardware_threads = 4
cpu_cache_L1 = 32KB
cpu_cache_L2 = 256KB
cpu_cache_L3 = 8MB

[scheduler]
scheduler_tick_freq = 4
; 0 es el valor por defecto del sistema operativo, cpu_clock_speed_Ghz
bucket_cgs_granularity = 100
; la cantidad de tramos fiscales (en cgs) en los que se divide la sociedad
max_budget = 30000
; presupuesto máximo (en cgs) asignable a un proceso
```

Los archivos de configuración deben estar en la carpeta `config`. Esta siempre tiene que tener el archivo `config.ini` por defecto dentro.

3.2. Parámetros de lanzamiento.

Los parámetros de lanzamiento se pueden combinar a la hora de ejecutar BancOS para sobrescribir la configuración del archivo .ini.

Sintaxis General

El orden de los argumentos no es relevante. La sintaxis general de ejecución es:

```
./bancos_kernel [OPCIONES] [PARÁMETROS HARDWARE] [PLANIFICACIÓN]
```

Configuración General :

Estos parámetros no se pueden añadir en el archivo .ini de configuración

- **-confile <archivo>**: Ruta al archivo .ini de configuración (Default: `conf/config.ini`).
- **-verbose**: Activa la salida detallada de depuración en consola (creación de procesos, cambios de estado, etc.). Invalida la consola de BancOS y solo se ven los mensajes del sistema.
- **-fgen <int>**: Frecuencia (en ticks) de generación de nuevos procesos aleatorios.

Parámetros de Hardware (Sobrescriben el archivo .ini):

- **-fcpu <float>**: Velocidad de reloj de todas las CPU (GHz).
- **-ncpu <int>**: Número de CPUs físicas, multiplica el número de cores.
- **-ncores <int>**: Número de núcleos por CPU.

- **-tcores <int>**: Hilos de hardware (threads) por núcleo.

Total hilos concurrentes = ncpu · ncores · tcores.

Planificación:

- **-schedticks <int>**: Intervalo de ticks del reloj entre cada ejecución del Scheduler.

Ejemplo

Simulación con hardware de 4 CPUs a 3.5Ghz, y frecuencia de planificación cada 5 ticks de reloj:

```
./bancos_kernel -fcpu 3.5 -ncpu 4 -schedticks 5
```

3.3. Consola de Comandos.

Durante la ejecución del simulador, BancOS mantiene activo un hilo de lectura sobre la entrada estándar (stdin). Esto permite al usuario interactuar con el kernel en tiempo real para inspeccionar el estado del sistema sin detener la simulación.

Los comandos disponibles son:

- **ps (Global Process List)**

- **Función:** Realiza un volcado completo de todos los procesos vivos en el sistema.
- **Detalles:** Recorre la lista doblemente enlazada global (desde `list_pcb_head`), mostrando el PID, estado y presupuesto de cada proceso, independientemente de si están en ejecución, listos o bloqueados.

```
<BANCOS_OS> ps

--- LISTA GLOBAL DE PROCESOS (5 activos) ---
PID: 383 | Budget: 280 | Estado: 2
PID: 915 | Budget: 35 | Estado: 2
PID: 386 | Budget: 18250 | Estado: 2
PID: 421 | Budget: 670 | Estado: 2
PID: 690 | Budget: 4820 | Estado: 2
```

- **run (Running Processes)**

- **Función:** Muestra exclusivamente los procesos que están consumiendo ciclos de CPU en este instante.
- **Detalles:** Itera sobre los núcleos (Cores) y hilos de hardware para identificar qué PID ocupa cada recurso de procesamiento. Es útil para verificar el paralelismo real.

```
<BANCOS_OS> run

=== PROCESOS EN EJECUCIÓN (CPU) ===
Core   | Hilo   | PID   | Budget   | Clase
-----|-----|-----|-----|-----
0      | 0      | 788   | 14540    | ALTA
0      | 1      | 226   | 13000    | ELITE
0      | 2      | 474   | 11100    | ALTA
0      | 3      | 34    | 16000    | ELITE
1      | 0      | 128   | 17500    | ELITE
1      | 1      | 965   | 16100    | ALTA
1      | 2      | 555   | 12140    | ALTA
1      | 3      | 871   | 13150    | ELITE
=====
```

- **qs (Queue Status)**

- **Función:** Inspecciona el estado del planificador CGS.
- **Detalles:** Visualiza la distribución de procesos en los distintos *Buckets* (estratos de presupuesto) y muestra la carga de la *RunQueue*. Permite observar en tiempo real cómo los procesos “caen” de clase social a medida que consumen su presupuesto.

```
<BANCOS_OS> qs
=== ESTADO DE LAS COLAS ===
Core 0: CPU [4/4] | Cola [61] | Inflación [x6]
Core 1: CPU [4/4] | Cola [59] | Inflación [x5]
=====
```

- **help:** Muestra la lista resumida de comandos disponibles.

```
<BANCOS_OS> help
Comandos disponibles:
ps    - Listar TODOS los procesos (Global List)
run   - Ver solo procesos en EJECUCIÓN (CPU)
qs    - Ver estado de colas y carga
exit  - Apagar el simulador
```

- **exit:** Fuerza una parada ordenada del simulador, liberando la memoria y destruyendo los mutex y variables de condición activos.

4. Conclusiones.

Gracias a BancOS se ha demostrado éxito en la simulación de una arquitectura de un sistema operativo funcional de manera concurrente y multihilo. Se ha implementado de manera robusta la separación de responsabilidades entre el *Clock*, el *Timer*, el *Scheduler* y el process generator, mediante mecanismos de sincronización POSIX.

Desde el punto de vista del rendimiento, la implementación de estructuras estáticas a **listas doblemente enlazadas intrusivas** ha sido determinante para el rendimiento del kernel, esto combinado con la política **CGS Capital Gains Scheduling**, mediante *buckets* y mapas de bits, ha permitido alcanzar una complejidad temporal de $O(1)$ en las operaciones críticas de selección y gestión de procesos.

Por último BancOS está implementado de manera modular lo que lo hace versátil para futuras ampliaciones, además ya implementa un sistema de simulación de interrupciones por hardware que se puede llamar desde cualquier parte de la simulación que lo requiera.

En definitiva BancOS demuestra que es posible implementar un planificador basado en conceptos económicos, que sea eficiente y escalable aunque tristemente demostrando como cuando un proceso “Rico” monopoliza la CPU, los procesos “Pobres” sufren escasez (inanición), reflejando la desigualdad de oportunidades.